Appendix: running the notebook on the DACHS cluster (step by step)

This appendix explains everything needed to run the notebook on the cluster DACHS, from the registration (nobody in the team has to know anything about it beforehand) to the download of the results. It was written while we did it for the first time, including the problems we ran into (table at the end).

Why a cluster at all? EfficientNet_B5 needs 456 x 456 pixel inputs and a lot of GPU memory. On a MacBook Air with 24 GB of unified memory (`mps`) a first run with batch size 16 filled the memory completely, the system swapped about 11 GB, and after 1.5 hours the run had not finished. DACHS is operated by the Hochschule Esslingen for the universities of applied sciences of Baden-Wuerttemberg and is free of charge for members of the shareholder universities, among them the HKA. A node of the queue `gpu1` has one NVIDIA L40S with 48 GB. That is also why the host names below end in `hs-esslingen.de` although we study at the HKA: only the machine is in Esslingen, the login name carries the prefix `hk_` of our university.

Overview. (A) register once, (B) connect (VPN, then `ssh`), (C) prepare the cluster once, (D) start the job, (E) bring the results back. Steps B to E are repeated for every run. Everything marked *terminal 1* happens in a shell that is logged in on the cluster, everything marked *terminal 2* in a normal, **local** terminal on your own computer.

A. One-time registration (about 30 minutes, once per person)

1. Open login.bwidm.de, choose *Hochschule Karlsruhe* and log in with the campus account (the same as for the RZ services). Under *Entitlement* the entry `bwDACHS` must be listed (it was for us without any request; if not, the Rechenzentrum has to grant it).
2. Under the registered services (page *Registry info*) register the service **DACHS**. Afterwards the page *Registry info: DACHS* shows `registration status: ACTIVE` and your **login name** `hk_<RZ id>`, for example `hk_abcd1234` (RZ id = the part of the campus mail address before `@hs-karlsruhe.de`, e.g. `abcd1234`). You need this name for every login.
3. On the same page click **Set Service Password** and choose a password for DACHS (it is only used for DACHS, not your campus password).
4. Create a **second factor** (TOTP): in the portal under *My Tokens* (`https://login.bwidm.de/user/twofa`) add a token and scan the QR code with an authenticator app. Without it a login is impossible. This token is **not** the one of the HKA VPN (see below).
5. Install the **FortiClient VPN** (download: `https://rz.h-ka.de/vpn`) for the connection from outside the campus (details in B.1). If you do not have the HKA second factor yet, the Rechenzentrum explains it in its *Introduction Handout*, chapter *Mehrfaktorauthentifizierung*.

B. Connect (for every session)

B.1 VPN. The login nodes are only reachable from the BelWue network: on the campus (eduroam) directly, otherwise through the **HKA VPN**. The settings in FortiClient VPN (*VPN konfigurieren*) follow the *Introduction Handout* of the Rechenzentrum, chapter *VPN-Einrichtung*; for the faculty IWI:

- VPN type *IPsec VPN*, connection name `VPN-HKA`, remote gateway `vpn.iwi.h-ka.de`, authentication method *key*, shared secret `IPsecFC_FakIWI`, XAuth *ask on login* (other faculties have their own gateway and secret, see the handout)
- expert settings, phase 1: IKE proposals AES128/SHA1 and AES256/SHA256, **DH group 18 only** (untick all others), key lifetime 86400 s, dead peer detection and NAT traversal on
- connect with `VPN-HKA`: user name = RZ id without `@h-ka.de`, password = the 6-digit code of the **HKA** authenticator (not the bwIDM one; wait until the code has changed)

Note: gateway and secret are marked as internal by the Rechenzentrum. The repository is private and only HKA members work in it; it must not be made public with this section.

B.2 Terminal 1: log in once and keep the connection open. The cluster asks for the one-time code of the **bwIDM** authenticator (`Your OTP:`) and then for the service password (`Password:`). The options keep the connection open for two hours, so that the second terminal needs no further codes:

```
ssh -o ControlMaster=auto -o ControlPath=~/.ssh/cm-%r@%h -o ControlPersis
```

At the first connection `ssh` shows the key fingerprint; compare it with the values on the DACHS login page (ED25519: `SHA256:X9eRJepYD3da3BM1pgiWxnvRc/Pt5eBLUr18tDUsZjU`) and answer `yes`. Afterwards the prompt looks like `[hk_abcd1234@login2 ~]$`. **Do not close this window.** Computations must never run on the login node, only through `sbatch` (below).

B.3 Terminal 2: the same connection for copying files. Open a *second local terminal* (not the one that is logged in on the cluster) and define the option once per terminal:

```
S="-o ControlPath=$HOME/.ssh/cm-%r@%h"
```

Every `ssh $S ...` and `scp $S ...` command in this terminal reuses the open connection without a new code.

C. One-time preparation of the cluster

C.1 Upload the folder (terminal 2; run it in the folder `Praktikum` of the repository; `scp` needs no backslash for the space in the folder name, only the quotes):

```
cd "<path to the repository>/Praktikum"
ssh $S hk_<RZ id>@dachs-login.hs-esslingen.de 'mkdir -p ~/ki-lab/data'
scp $S -r "Aufgabe 4" hk_<RZ id>@dachs-login.hs-esslingen.de:~/ki-lab/
```

C.2 Create the Python environment and fetch data and weights (terminal 1, on the **login node**, because compute nodes may have no internet; the cluster provides Python only as a module, there is no `uv`):

```
module load devel/python/3.13.3-gnu-14.2          # check with: module av
cd ~/ki-lab
[ -d .venv ] || python -m venv .venv
source .venv/bin/activate
pip install --no-cache-dir torch torchvision pandas scipy matplotlib nbco
cd "Aufgabe 4"
python -c "import torchvision; [torchvision.datasets.Flowers102('../data'
python -c "from torchvision.models import *; resnet18(weights=ResNet18_We
```

`pip` installs a PyTorch build for CUDA 13 (several GB, a few minutes); the driver of the GPU nodes (595, CUDA 13.2) supports it. The environment and the data stay in `~/ki-lab`, so this step is needed only once.

C.3 Write the job script (terminal 1, in `~/ki-lab/Aufgabe 4`):

```
cat > job.sh <<'EOF'
#!/bin/bash
#SBATCH --partition=gpu1
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64gb
#SBATCH --time=02:00:00
#SBATCH --job-name=ki-lab-a4
#SBATCH --output=slurm-%j.out
module load devel/python/3.13.3-gnu-14.2
source ~/ki-lab/.venv/bin/activate
cd ~/ki-lab/"Aufgabe 4"
python -c "import torch; print('CUDA available:', torch.cuda.is_available
jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor
EOF
```

D. Start the job and watch it

```
sbatch job.sh                    # submits the job, prints "Submitted batch
squeue -u $USER                  # ST = PD: waits for a free GPU, R: runnin
squeue --start -u $USER          # expected start time while it waits
cat $(ls -t slurm-*.out | head -1)    # output of the job; first lines: GP
scancel <id>                     # cancel a job
sacct -u $USER --format=JobID,JobName%20,State,Elapsed,MaxRSS --starttime
```

The file `slurm-<id>.out` exists only after the job has started. When `squeue -u $USER` shows nothing any more, the job is finished (`sacct` shows `COMPLETED` or `FAILED`). The executed notebook replaces `assignment_04.ipynb` in `~/ki-lab/Aufgabe 4` and the run writes its machine file `results/<user>_Linux.json` (section 7).

E. Bring the results back

Terminal 2 (local), in the folder `Praktikum`:

```
mkdir -p "Aufgabe 4/results"
scp $S hk_<RZ id>@dachs-login.hs-esslingen.de:"~/ki-lab/Aufgabe 4/results
scp $S hk_<RZ id>@dachs-login.hs-esslingen.de:"~/ki-lab/Aufgabe 4/assignm
```

Commit the new file in `results/` so that the machine comparison in section 7 contains the cluster as well (`git add "Aufgabe 4/results"`). The executed notebook from the cluster is stored under a different name so that the local version is not overwritten; do not commit both versions.

Log out when you are done: `exit` in terminal 1 (otherwise the connection closes itself after two hours).

Problems we ran into, and what solved them

Problem	Cause	Solution
Run on the Mac does not finish, system swaps, kernel dies	EfficientNet_B5 at 456 px with batch size 16 needs more than the 24 GB of unified memory	Batch size 8 on every machine; the full run on the cluster GPU (48 GB)
`ssh` to `hs-esslingen.de` although we study at the HKA	DACHS is operated by the Hochschule Esslingen, the HKA is one of the shareholders	Correct: only the host name is Esslingen, the login name carries the prefix `hk_`
VPN: *pre-shared key wrong*	Not the key, but the IKE settings: the expert settings of the FortiClient profile need **only DH group 18** (other groups unticked)	Set the DH group exactly as in the RZ handout, then the key is accepted
Commands pasted into the cluster shell fail (`No such file or directory`, host-key question)	Upload commands belong into a second, *local* terminal	Keep the cluster session open in terminal 1, run `scp` in terminal 2
`scp: dest open "ki-lab/Aufgabe\ 4/": Failure`	`scp` (SFTP mode) takes the path literally: a backslash before the space becomes part of the folder name	Quote the path and write the space plainly: `"~/ki-lab/Aufgabe 4/"`
`uv: command not found`	The cluster module provides Python only	`python -m venv` and `pip install` instead of `uv`
`ModuleNotFoundError: No module named 'scipy'` when loading Flowers102	torchvision reads the label files with `scipy.io`	`pip install scipy` (also in `requirements.txt`)
`Permission denied: .venv/bin/Activate.ps1` when the setup is run a second time	`python -m venv` tries to overwrite the existing environment	Create it only if it does not exist: `[ -d .venv ] || python -m venv .venv`
Job status `PD` and no `slurm-*.out` file right after `sbatch`	The job waits in the queue for a free GPU; the output file exists only after it started	Check `squeue -u $USER` until `R` appears (a few minutes), then read the output file
Is PyTorch from `pip` compatible with the GPU driver?	`pip install torch` pulls CUDA 13 wheels; the DACHS driver 595 supports CUDA 13.2	The job prints `CUDA available: True` first; with an older driver install a CUDA 12 build instead
One-time code is rejected	The code is valid for 30 seconds and the VPN uses a different token (HKA) than the cluster (bwIDM)	Wait for the next code and use the right authenticator entry
The commands from this appendix show `<RZ id>`	Placeholder for your own login name	Replace it with `hk_` plus your RZ id (see A.2)

Result of the runs. On the cluster the summed training time of all approaches was about 4 minutes (notebook total a few minutes), on the MacBook Air (`mps`) the summed training time was 32 minutes and the whole notebook took 73 minutes (EfficientNet_B5 with fine-tuning alone 23 minutes), where the first attempt on the laptop with batch size 16 had not finished after 1.5 hours.